



Universidad
Nacional de
General
Sarmiento

Instituto de Industria
Licenciatura en Sistemas

TRABAJO FINAL

Sistemas Operativos y Redes 2

Primer Semestre 2026

"Eficacia y costo computacional de los modelos Mistral-7B y Phi-3 en la clasificación semántica de auditoría interna frente a amenazas multi-paso "

Alumno/a	Santiago Agustín Gonzalez
Legajo	42841155
Docente	Benjamín Chuquimango
Fecha de entrega	19/06/2026

Modalidad Individual — Obligatorio

Buenos Aires, Argentina — 2026

Índice de contenidos

Índice de contenidos	2
1. Resumen ejecutivo	3
2. Planteamiento del problema	4
3. Pregunta de investigación y objetivos	5
3.1 Pregunta de investigación	5
3.2 Objetivo general	5
3.3 Objetivos específicos	5
4. Marco teórico	6
5. Metodología	10
5.1 Enfoque metodológico	10
5.2 Herramientas y justificación	10
6. Implementación y desarrollo	12
6.1 Diseño de la solución	12
6.2 Implementación por componente	13
6.3 Análisis según modelo OSI o capas del SO	19
6.4 Pruebas y verificación	21
7. Resultados y análisis	23
7.1 Comparación con los objetivos	23
7.2 Limitaciones identificadas	25
7.3 Propuestas de mejora y trabajo futuro	26
8. Conclusiones y trabajo futuro	27
9. Referencias bibliográficas	28
10. Anexos	29
Anexo A — [Repositorio de Github]	29
Anexo B — [Capturas complementarias de actividades rutinarias]	29

1. Resumen ejecutivo

El análisis de registros de auditoría a bajo nivel en sistemas operativos Linux mediante herramientas nativas como ***auditd*** representa un pilar fundamental para la reconstrucción forense de incidentes. Sin embargo, la estructura fragmentada y secuencial de esta telemetría, sumada a las limitaciones sintácticas de las herramientas tradicionales basadas en expresiones regulares (Regex), genera deficiencias críticas: una elevada tasa de falsos positivos en entornos administrativos rutinarios y la incapacidad absoluta de correlacionar ataques coordinados multi-paso en ventanas temporales determinadas. El presente trabajo de investigación aborda este problema mediante el diseño de un entorno experimental controlado para evaluar la eficacia analítica y el costo computacional de dos modelos de lenguaje extensos (LLMs) de ejecución local: Mistral 7B y Phi-3 Mini. Metodológicamente, se estructuró una matriz de verdad de campo segregada en ventanas de tres minutos para simular escenarios de intrusión, mantenimiento legítimo y ruido cotidiano. Los resultados demuestran que, frente al análisis rígido y aislado de la Regex, los modelos lógicos poseen la capacidad semántica de correlacionar llamadas al sistema (*syscalls*) y metadatos de usuario en el tiempo, identificando con precisión escaladas de privilegios. No obstante, bajo restricciones físicas de hardware comercial (GPU de 4 GB), la inferencia local híbrida genera un cuello de botella en la CPU que eleva las latencias a promedios de entre 2.5 y 3.5 minutos por bloque. Se concluye que, aunque la latencia actual descarta a los LLMs para mitigación en tiempo real, su riqueza interpretativa los consolida como herramientas revolucionarias para la automatización de auditorías forenses diferidas (*offline*).

Palabras clave: [Auditoría de Sistemas], [Modelos de Lenguaje Locales], [Ataques Multi-Paso], [Linux Auditd], [Ciberseguridad Forense]

2. Planteamiento del problema

En el panorama actual de la ciberseguridad en sistemas operativos la recopilación de auditorías a bajo nivel es una práctica indispensable para la detección de intrusiones y la reconstrucción forense de incidentes. En entornos Linux el subsistema de auditorías **auditd** es el estándar en la monitorización de eventos críticos del sistema, tales como llamadas al sistema (System Calls), modificaciones en archivos, esenciales de configuración y escalaciones de privilegios. Sin embargo, la implementación de esta tecnología presenta un quiebre crítico en la fase de análisis de datos dentro del espacio de usuario. El problema fundamental es que **auditd** genera un volumen masivo de registros en formato de texto crudo altamente fragmentado, secuencial y con una alta complejidad para la persona que lo lee. Cada acción ejecutada por un proceso o usuario es documentada en un log de múltiples líneas. Los mecanismos tradicionales de filtrado, basados en herramientas auditorías como ausearch o en clasificadores estáticos mediante expresiones regulares (regex) operan registro por registro generando dos grandes deficiencias técnicas: por un lado, una elevada tasa de falsos positivos ante operaciones administrativas legítimas pero inusuales; por otro lado, la incapacidad de correlacionar ataques multi-paso complejos, donde eventos que individualmente se categorizan como “bajo” o “medio” constituyen, al analizarse en una ventana temporal determinada, una intrusión crítica en desarrollo.

Estas deficiencias las sufren los administradores de sistemas (SysAdmins), ingenieros de confiabilidad de sitios (SRE) y analistas de Centros de Operaciones de Seguridad (SOC). Estos profesionales se enfrentan diariamente al fenómeno conocido como “fatiga de alertas”, donde la cantidad abrumadora de alertas, de las cuales muchas son de baja prioridad, generan un estado de agotamiento mental y operativo que dificulta el procesamiento reflexivo de la información. Al verse expuestos a señales repetitivas y no urgentes, comienzan a ignorarlas lo cual puede ser fatal.

Las consecuencias económicas y operativas de estas deficiencias metodológicas son severas. Según el estudio global corporativo *Cost of a Data Breach Report 2025* realizado por IBM Security y el Ponemon Institute, el tiempo promedio que le toma a una organización identificar y contener una brecha de seguridad es de 241 días (181 días para la identificación y 64 días para la contención). El mismo informe revela que el costo promedio de una filtración de datos a nivel empresarial ha alcanzado los 4.44 millones de dólares, impulsado principalmente por las multas regulatorias elevadas y mayores costos de detección y escalamiento en Estados Unidos. Para dar a los defensores velocidad y escalabilidad las organizaciones empezaron a adoptar el uso de IA y automatización en la seguridad, generando una brecha de 3.62 millones de dólares, en comparación con los 5.52 millones de dólares de aquellas que no las usan, esto es un ahorro crítico de 1.9 millones de dólares por incidente. Además, el uso extensivo de IA y automatización redujo el tiempo para identificar y contener una brecha de seguridad en 80 días (pasando de un promedio global de 284 sin IA a 204 días con IA masiva).

Vale la pena medir el tiempo y la precisión de Modelos de Lenguaje de Gran Escala (LLM) de código abierto corriendo en entornos estrictamente locales (como *mistral:7b* o *phi3:mini* mediante arquitecturas de inferencia mixta) que permite dotar al pipeline de auditoría de una capacidad de abstracción contextual y razonamiento secuencial que las expresiones regulares tradicionales no poseen. Al agrupar los logs crudos en ventanas temporales específicas, la inteligencia artificial puede interpretar la cronología completa de las llamadas al sistema, aislar el comportamiento malicioso con precisión y reducir drásticamente los falsos positivos sin comprometer la privacidad ni la soberanía de los datos sensibles de la infraestructura. Reducir la brecha de detección de días a escasos segundos mediante un análisis semántico local justifica con creces el diseño y desarrollo de una solución automatizada.

3. Pregunta de investigación y objetivos

3.1 Pregunta de investigación

¿Puede el modelo `mistral:7b`, utilizando una estrategia de prompting de Cadena de Pensamiento (Chain-of-Thought), detectar ataques multi-paso mediante el análisis de logs de auditoría agrupados en ventanas temporales, con mayor precisión y menor tasa de falsos positivos que un clasificador tradicional basado en expresiones regulares?

3.2 Objetivo general

Comparar la velocidad y precisión de los modelos de LM, `mistral:7B` y `phi3:mini` como IAs locales para la auditoría y la seguridad enfocado en ataque multi-paso.

3.3 Objetivos específicos

1. Diseñar un entorno de control y métricas base.

Construir una matriz de verdad de campo (Ground Truth) categorizada en niveles de riesgo basada en eventos nativos de auditoría para ventanas temporales de 3 minutos.

2. Automatizar el flujo de evaluación.

Desarrollar un pipeline en Python que integre el procesamiento de los DataSets locales con la API de inferencia de modelos de lenguaje para su calificación secuencial.

3. Cuantificar la viabilidad de la infraestructura local.

Medir los tiempos de procesamiento híbrido (CPU/GPU) de los modelos `mistral:7b` y `phi3:mini` al procesar contextos extensos bajo restricciones de hardware estándar.

4. Contrastar la eficacia analítica.

Comparar las tasas de falsos positivos y la capacidad de detección semántica de los LLMs frente a un clasificador tradicional basado en expresiones regulares (Regex).

4. Marco teórico

El presente marco teórico establece los fundamentos conceptuales que respaldan la viabilidad de utilizar inteligencia artificial generativa, específicamente Modelos de Lenguaje Extensos (LLMs) de ejecución local, frente a clasificadores tradicionales (Expresiones Regulares) en el ámbito de la auditoría forense de sistemas operativos. A través del análisis de fuentes recientes, se expone la transición metodológica desde la detección sintáctica de intrusiones hacia la comprensión semántica y temporal de amenazas complejas.

Evolución de la detección de instrucciones:

Históricamente, los Sistemas de Detección de Intrusiones Basados Host (HIDS) han dependido de firmas estáticas y expresiones regulares para identificar comportamientos maliciosos en los sistemas operativos (logs). Sin embargo la sofisticación de los ataques modernos ha evidenciado la fragilidad de este enfoque analítico.

El problema central de los métodos tradicionales radica en su “ceguera semántica”. Una expresión regular evalúa líneas de texto de forma aislada, buscando coincidencias de caracteres precisos (como la invocación de la regla **-k passwd_changes** en **auditd**), sin la capacidad de interpretar el contexto o la intencionalidad de la acción. Esto genera un doble problema operativo documentado en la industria: una alta tasa de falsos negativos ante técnicas de ofuscación y una fatiga de alertas masiva generada por falsos positivos ante actividades rutinarias administrativas.

Para abordar esta deficiencia, la investigación contemporánea ha comenzado a integrar Modelos de Lenguaje Extensos (LLMs) en el procesamiento de logs, dada su capacidad nativa de entender el lenguaje natural y las secuencias de código. En este contexto, Guan et al. (2024) propone el framework LogLLM, enfocado en la detección de anomalías extrayendo vectores semánticos de los mensajes de log del sistema.

- Este trabajo fundamenta teóricamente el reemplazo del análisis tradicional. Demuestra empíricamente que los LLMs son capaces de abstraer el significado detrás de comandos crudos y no estructurados, superando con precisión a algoritmos tradicionales.
- Pero, la investigación de Guan et al. (2024) presenta limitaciones con respecto al presente proyecto ya que propone una arquitectura dual sumamente pesada (combinando redes BERT con modelos Llama) que procesa anomalías de software en general. Esta dependencia de múltiples redes neuronales requiere recursos de hardware masivos, lo que la hace técnica y operativamente inviable para la auditoría local en hardware restringido planteada en este trabajo, alejándose además del seguimiento estricto de llamadas al sistema (Syscalls) para intrusiones en Linux.

Profundizando en la transición metodológica hacia la Inteligencia Artificial, Palma et al. (2025) analizan cómo los LLMs logran superar incluso a los algoritmos de Machine Learning tradicional (como XGBoost o Random Forest) en el análisis de registros de ciberseguridad.

- La investigación valida uno de los pilares de este trabajo: la “explicabilidad” forense. Palma et al. (2025) demuestran que los LLMs no sólo predicen el

riesgo con mayor precisión, sino que ofrecen un razonamiento estructurado (Chain-of-Thought) que explica el vector de amenaza. Esto avala metodológicamente la extracción del campo “razonamiento” en el pipeline automatizado diseñado para `mistral:7b` y `phi3:mini`.

- Sin embargo delimita a la detección estática de vulnerabilidades de software (código estático) utilizando modelos enfocados a la nube o de mayor peso (Llama 3.2). Carece de un enfoque dinámico que evalúe la correlación de eventos cronológicamente dispersos en el espacio del núcleo (Kernel Space) durante un incidente en tiempo real.

La complejidad de los ataque multi-paso en la infraestructura del SO:

A diferencia de un ataque directo (como la ejecución de un *exploit* aislado), un Amenaza Persistente Aislada (APT) o un ataque multi-paso se define como una secuencia cronológica de eventos de bajo perfil que, analizados individualmente, no representan una violación crítica de seguridad. Una secuencia típica involucra fases de reconocimiento interactivo (**Is, whoami**), reintentos de escalada de privilegios rebotados (**sudoers_changes** fallidos) y, finalmente, un compromiso exitoso (**UID=0** operando sobre **/etc/shadow**).

Esta naturaleza distribuida en el tiempo necesita que la herramienta de auditoría posea una “ventana de contexto” lo suficientemente amplia como para recordar la primera acción del atacante y correlacionarla con la última.

En su reciente revisión sobre ciberseguridad e IA, Xu et al. (2024) identifican que el uso de LLMs está migrando de la ejecución de tareas aisladas hacia la orquestación de flujos de trabajo de seguridad complejos mediante agentes autónomos.

- Este estudio convalida la viabilidad cognitiva de los LLMs para correlacionar eventos temporales. Establece que los modelos de lenguaje modernos tienen la retención de contexto necesaria para interpretar las “fases” de un ciberataque de manera integral.
- Aún así, al ser una Revisión Sistemática de Literatura (SLR), el documento identifica la necesidad de crear modelos más interpretables frente a ataques multi-paso, pero no diseña una solución arquitectónica implementable, ni aborda el dilema físico del embotellamiento del procesador (CPU) que enfrentan estas tecnologías fuera del laboratorio.

Para aterrizar estos conceptos en la infraestructura del host, Sun et al. (2025) desarrollaron SHIELD, un framework que integra LLMs específicamente para la detección de intrusiones basadas en host (HIDS). Su trabajo destaca la dificultad inherente de usar IA en este campo debido a “los ruidos de fondo” (eventos administrativos rutinarios) que confunden al modelo y diluyen los recorridos reales del ataque.

- La investigación de Sun et al. (2025) aporta la justificación teórica sobre por qué un LLM genera falsos positivos ante comandos inofensivos. Demuestra que el modelo sufre de hipervigilancia al procesar logs crudos, consolidando la necesidad empírica de generar ventanas temporales filtradas (como las implementadas en este proyecto) para aislar la actividad de la red y las syscalls críticas de **auditd**.

- El problema es que para resolver la confusión del modelo, SHIELD emplea Autoencoders enmascarados y un proceso de Aumento Determinista de Datos (DDA). Estas complejas canalizaciones de pre-procesamiento requieren capacidades de cómputo en clúster, lo que resulta prohibitivo para la propuesta de valor de esta investigación, la cual prioriza un enfoque directo y ligero de inferencia híbrida adaptado a infraestructura de hardware del consumidor.

Limitaciones físicas y viabilidad operativa de los LLMs locales (Edge Computing):

La transferencia del análisis cognitivo desde la nube hacia la infraestructura local (Edge Computing) o servidores locales On-Premise es un requerimiento ineludible en el ámbito de la ciberseguridad gubernamental o corporativa por estrictas políticas de privacidad. Sin embargo, la ejecución local impone severas restricciones de memoria VRAM, lo que obliga a la delegación parcial de capas de inferencia neuronal hacia el procesador central (CPU).

La evaluación del balance entre rendimiento de hardware y capacidad analítica, Kumar (2025) realizó un benchmarking sobre la seguridad y eficiencia térmica de modelos locales alojados en equipos con 16 a 32 GB de RAM, posicionando a Mistral 7B y a Phi-3 3.8B como los líderes indiscutidos.

- El trabajo de Kumar (2025) proporciona el respaldo métrico exacto que justifica la selección metodológica de los modelos evaluados en este trabajo. Confirma empíricamente que tanto Mistral 7B como Phi-3 pueden ejecutarse eficientemente de manera local manteniendo una integridad analítica sobresaliente, optimizando el consumo de recursos sin sacrificar la privacidad.
- La limitación de la del benchmark de Kumar es que se enfoca exclusivamente en la generación y análisis de código fuente en desarrollo de software (Python). Queda un vacío investigativo profundo --que este trabajo viene a llenar-- respecto a cómo esta eficiencia y velocidad se trasladan a una tarea forense semántica sobre logs del Kernel de Linux, donde los cuellos de botella del KV Cache sobrecargan la CPU de forma distinta.

Finalmente, para comprender la superioridad técnica en la detección técnica de secuencias lógicas complejas a pesar de las limitaciones de hardware, es vital remitirse al Reporte Técnico oficial de la arquitectura de Phi-3 elaborado por Abdin et al. (2024) para Microsoft.

- En el Reporte Técnico los autores explican que la extraordinaria capacidad de razonamiento deductivo de Phi-3 Mini (que compite con modelos masivos a pesar de tener solo 3.8 billones de parámetros) se debe a un entrenamiento en un DataSet altamente curado 3.3 billones de tokens sintéticos orientados a la lógica. Este aporte arquitectónico explica teóricamente por qué Phi-3 mini fue el único capaz de hilvanar los metadatos (UIDs y Syscalls) para detectar exitosamente el ataque multi-paso en la simulación experimental.
- Al tratarse de un reporte técnico fundacional, el modelo se presenta como un LLM de propósito y razonamiento general. Al no contar con un proceso de alineación (fine-tuning) exclusivo para el dominio de la seguridad defensiva, se explica y valida el sesgo observado durante las pruebas, donde su extrema agudeza lógica malinterpretó el mantenimiento normal del sistema (**V-ADM-01**) catalogandoló como una intrusión crítica.

En síntesis, la literatura reciente 2024-2025 demuestra un consenso científico claro: mientras que los enfoques basados en Expresiones Regulares son insuficientes para el dinamismo de los ataques modernos, los Modelos de Lenguaje Locales representan una alternativa forense superior gracias a su capacidad de retención de contexto. No obstante, los estudios revelan que la implementación arquitectónica de estas tecnologías en hardware comercial aún carece de evaluaciones de rendimiento específicas para cargas de trabajo de **auditd**, posicionando a la presente investigación como un puente empírico entre la eficacia semántica teórica y la viabilidad técnica real.

5. Metodología

5.1 Enfoque metodológico

Para evaluar los modelos, se diseñaron 5 escenarios de control (ventanas de 3 minutos) simuladas y una matriz de Ground Truth categorizadas en tres niveles de riesgo “BAJO”, “MEDIO” y “ALTO”. El Ground Truth define la referencia para evaluar la precisión y velocidad de los modelos de LLM (Mistral 7b y Phi-3 mini) frente al razonamiento y clasificación de riesgo de cada ventana, las cuales se dividen según el nivel de privilegio que posee el usuario o atacante y el tipo de actividades que realiza.

Las ventanas a analizar serán las siguientes:

- **V-ATK:** Que representa el ataque multi-paso, con una clasificación de riesgo “ALTO”.
- **V-ADM:** Que representa el comportamiento de un administrador del sistema realizando tareas de mantenimiento, con una clasificación de riesgo “MEDIA”.
- **V-NRM:** Que representa actividad estándar del usuario común sin privilegios, como la lectura de archivos o la conexión a páginas web. Tiene una clasificación de riesgo “BAJA”.

id_ventana	Clasificación esperada
V-ATK-01	ALTO
V-ADM-01	MEDIO
V-NRM-01	BAJO
V-NRM-02	BAJO
V-NRM-03	BAJO

Tabla 1. Ground Truth inicial. Fuente: Elaboración propia.

Esta metodología es la más adecuada para la comparación de velocidad y precisión de modelos locales de IA en auditoría y seguridad ya que les permite demostrar su capacidad de razonamiento al tener una ventana completa de 3 minutos que contiene no solo el proceso objetivo sino que también un poco de ruido rutinario. Este ruido, que los modelos tendrán que racionalizar, ayudará a observar cómo las IAs llegan a resultados que son falsos positivos/negativos.

5.2 Herramientas y justificación

Herramienta / Tecnología	Propósito en el trabajo	Justificación de la elección
Oracle Virtual Box	Se usó como medio de despliegue del entorno víctima virtualizado.	Se eligió esta herramienta para aislar el entorno de pruebas y garantizar que el hardware físico del host permanezca íntegro y seguro. Además, asegura que

Herramienta / Tecnología	Propósito en el trabajo	Justificación de la elección
		la telemetría generada por auditd esté libre de contaminación de procesos del sistema anfitrión.
LM Studio	Se usó como servidor de inferencia para la ejecución de los modelos de LLM (Mistral 7b y Phi-3 mini) en el hardware del host físico.	Se implementó LM Studio y no otras herramientas ya que está construido sobre el motor llama.cpp que integra soporte para Vulkan. En cambio, al intentar usar Ollama no detectaba la GPU utilizada para correr los modelos de LLM porque necesitaba un driver en específico para cómputo llamado ROCm, de el cual la GPU era incompatible al tener una arquitectura Polaris que ya no cuenta con soporte.
Python 3.10.12	Se adoptó como lenguaje de programación central para el desarrollo del pipeline de evaluación automatizado.	Se eligió Python ya que es el estándar para tareas de ciencias de datos y ciberseguridad. Su ecosistema de librerías nativas facilitó la lectura estructurada de los DataSets, la evaluación del enfoque tradicional (Regex) y la comunicación HTTP para inyectar las ventanas temporales en el entorno de inferencia de IA, todo bajo un flujo de ejecución cronometrado.
Linux Audit Framework (auditd)	Se utilizó como el sistema principal de recolección de telemetría y eventos de seguridad dentro de la máquina virtual.	Se la eligió porque puede operar interceptando syscalls directamente en la frontera entre el espacio del usuario y el espacio del núcleo. Es indispensable ya que asegura que los eventos como execve u openat sean inmutables, estructurados y reflejen con precisión el comportamiento real del atacante, algo que una simple captura de red no podría lograr.
Ubuntu Server	Se implementó como el entorno base sobre el cual se simulaban los escenarios de ataque y el ruido cotidiano.	La elección de una distribución orientada a servidores sin interfaz gráfica responde a la necesidad de replicar un entorno de producción realista. Las infraestructuras corporativas suelen utilizar servidores de este tipo, con acceso remoto mediante SSH, lo que le otorga validez y aplicabilidad directa a los resultados del experimento.

Tabla 2. Herramientas y Justificación. Fuente: Elaboración propia.

6. Implementación y desarrollo

6.1 Diseño de la solución

Para llevar a cabo la evaluación de los modelos de lenguaje frente al análisis de telemetría se diseñó una arquitectura de laboratorio virtualizada y segmentada. El objetivo de este diseño modular es aislar el entorno de generación de amenazas (víctima) del entorno de procesamiento cognitivo, replicando las condiciones de un despliegue de seguridad real (Edge Computing).

Como se observa en la Figura 1 la arquitectura se divide en dos componentes principales:

1. **Entorno víctima (Máquina Virtual):**

Se desplegó una Máquina Virtual utilizando Oracle VirtualBox, equipada con el sistema operativo Ubuntu Server. En este entorno se ejecutaron los comandos correspondientes para simular los escenarios de ataque y actividades rutinarias. El Linux Audit Framework (**auditd**) actuó como el sensor principal recolectando las llamadas al sistema y describiéndolas en texto plano. En este mismo entorno funcionan los pipelines automatizados para cada técnica de análisis (Mistral 7b, phi-3 mini y Regex) desarrollados en Python). Estos scripts son responsables de filtrar los logs para reducir la cantidad de líneas totales, enfocándose en las líneas que dispararon las reglas que se definieron en auditd, y limitar la cantidad de líneas leídas a 30 para evitar saturar los tokens de cada modelo (9192 para cada modelo). Obviamente el evaluador Regex también se limita a leer solo 30 líneas para mantener la consistencia metodológica. Finalmente, los scripts general el prompt para los modelos y empaquetan los datos para su envío.

2. **Entorno de inferencia local (Host Físico):**

Para evaluar los cuellos de botella del hardware comercial, la clasificación semántica se delegó al host físico anfitrión (equipado con un procesador AMD Ryzen 5 1600 six-core, una GPU Radeon RX 570 de 4GB de VRAM y 16 GB de RAM). Se utilizó la plataforma LM Studio para exponer una API local (puerto 1234) compatible con el estándar de OpenAI. A través de este servidor de inferencia, se ejecutaron los modelos Mistral 7B Instruct v0.3 QA_K_M GGUF, un modelo pesado (4.84GB) de 7 billones de parámetros y phi-3.1-mini-128k-instruct QA_K_M GGUF un modelo un poco más liviano (2.33GB) de 4 billones de parámetros.

La integración entre ambos entornos se resolvió mediante peticiones HTTP estructuradas. El script de Python en la máquina virtual inyecta los payloads (ventanas de logs y prompts de sistema) hacia la API del host físico. Una vez que LM Studio procesa la información distribuyendo la carga de capas entre la CPU y la GPU (debido a las restricciones de los 4GB de VRAM), retorna un objeto JSON con el veredicto del riesgo y el razonamiento forense.

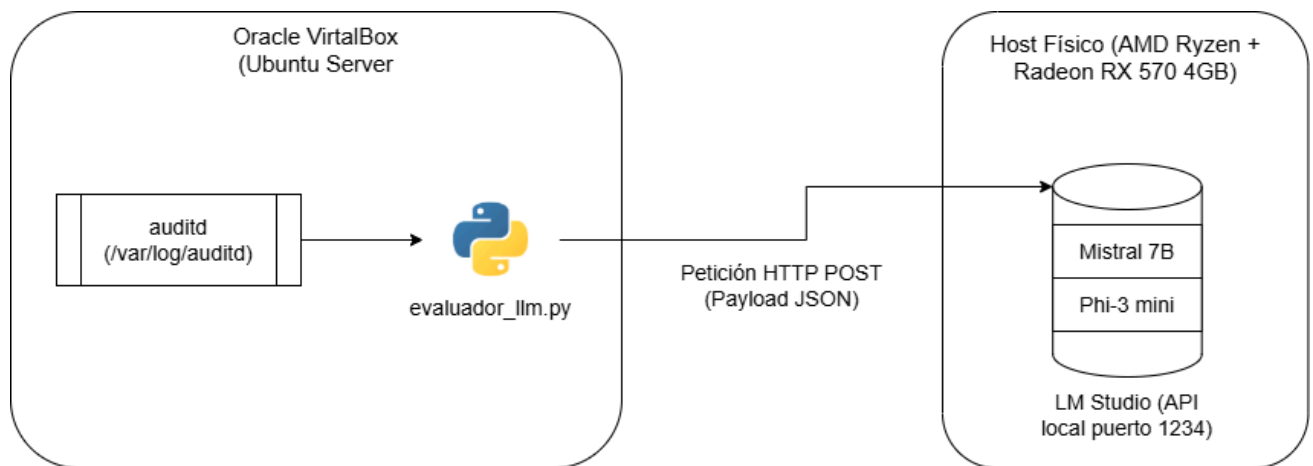


Figura 1. Diagrama de Arquitectura. Fuente: Elaboración propia

6.2 Implementación por componente

La arquitectura general descrita previamente se materializa a través de cinco componentes técnicos configurados específicamente para este experimento. A continuación se detallan los parámetros y la lógica de implementación de cada uno.

1. Sensor de telemetría (Reglas de auditd):

El punto de origen de los datos es el daemon de auditoría de Linux. En lugar de capturar la totalidad de los eventos del sistema, se implementó un archivo de configuración específico (**custom.rules**) diseñado para detectar vectores críticos del ataque.

```

alumno@alumno:~$ sudo bash -c "cat << 'EOF' > /etc/audit/rules.d/custom.rules
# Regla 1: acceso a /etc/passwd
-w /etc/passwd -p rwa -k passwd_changes

# Regla 2: modificaciones a /etc/sudoers
-w /etc/sudoers -p rwa -k sudoers_changes

# Regla 3: ejecucion de comandos como root
-a always,exit -F arch=b64 -S execve -F uid=0 -k root_commands

# Regla 4: acceso a /etc/shadow
-w /etc/shadow -p rwa -k shadow_access
EOF"
  
```

Figura 2. Reglas de auditd definidas en custom.rules. Fuente: Elaboración propia.

Se establecieron reglas con identificadores (claves **-k**) para monitorear intentos de modificación en archivos de credenciales (**-w /etc/passwd -p rwa -k passwd_changes** y **/etc/shadow**), alteraciones en permisos (**/etc/sudoers**) y la ejecución de binarios de 64 bits con privilegios de superusuario (**-S execve -F uid=0 -k root_commands**). Esto garantiza que el DataSet base contenga las syscalls críticas necesarias para el análisis.

2. Simulaciones de ventanas de ataque y actividades rutinarias:

Para armar los DataSets que los modelos van a procesar se simularon 5 ventanas de 3 minutos cada una con actividades que presentan distintos niveles de riesgo. A continuación se describen las claves auditd involucradas y el proceso de simulación de cada ventana:

ID Ventana	Duración	Eventos incluidos (Claves de auditd involucradas)	Clasificación esperada (Ground Truth)
V-ATK-01	3 min	exec_commands (reconocimiento) → sudoers_changes (success=no) → shadow_changes / passwd_changes (success=yes, euid=0)	ALTO
V-ADM-01	3 min	exec_commands → sudoers_changes con uid=0 (modificación legítima y limpia por administrador)	MEDIO
V-NRM-01	3 min	exec_commands estándar de usuario común (sin llamadas privilegiadas ni accesos a rutas críticas)	BAJO
V-NRM-02	3 min	Intentos de lectura legítimos/autorizados sobre archivos del sistema (sin flags de escritura o alteración)	BAJO
V-NRM-03	3 min	Comandos de usuario rutinarios de 64 bits disparando únicamente la clave exec_commands de manera aislada	BAJO

Tabla 3. Ventanas simuladas con claves auditd. Fuente: Elaboración propia.

- **V-ATK-01:** ventana del ataque

Minuto 0:00 (Acción 1 - Reconocimiento): Ejecución de comandos de exploración de usuario.

```
whoami \&\& id \&\& uname -a
```

Minuto 1:15 (Acción 2 - Intento Fallido): Se simula la inyección maliciosa en el archivo de privilegios. Se busca un "Permiso denegado".

```
echo "alumno ALL=(ALL) NOPASSWD:ALL" >> /etc/sudoers
```

Minuto 2:30 (Acción 3 - Escalada y Compromiso): Eleva privilegios y simula modificar las credenciales.

```
sudo su
```

Una vez que se ve que el prompt cambió a `root@...` se ejecuta:

```
touch /etc/shadow \&\& exit
```

Extracción (Antes de llegar al minuto 3:00): Guardo el registro crudo.

```
sudo ausearch --start recent --raw > V-ATK-01.txt
```

```

GNU nano 6.2                                V-ATK-01.txt                                S
UID="root" OGID="root"
type=PROCTITLE msg=audit(1780773394.216:1329): proctitle=7375646F006175736561726368002D2D73746172740
0726563656E74002D2D726177^]
type=SYSCALL msg=audit(1780773394.216:1330): arch=c000003e syscall=257 success=yes exit=13 a0=ffffff
9c a1=7f51a077e2ac a2=80000 a3=0 items=1 ppid=1336 pid=1366 auid=1000 uid=1000 gid=0 euid=0 suid=0 f
suid=0 egid=0 sgid=0 fsgid=0 tty=pts0 ses=6 comm="sudo" exe="/usr/bin/sudo" subj=unconfined key="pas
swd_changes" ^]ARCH=x86_64 SYSCALL=openat AUID="alumno" UID="alumno" GID="root" EUID="root" SUID="roo
t" FSUID="root" EGID="root" SGID="root" FSGID="root"
type=CWD msg=audit(1780773394.216:1330): cwd="/home/alumno" ^]
type=PATH msg=audit(1780773394.216:1330): item=0 name="/etc/passwd" inode=1050192 dev=08:02 mode=010
0644 ouid=0 ogid=0 rdev=00:00 nametype=NORMAL cap_fp=0 cap_fi=0 cap_fe=0 cap_fver=0 cap_frootid=0 ^]O
UID="root" OGID="root"
type=PROCTITLE msg=audit(1780773394.216:1330): proctitle=7375646F006175736561726368002D2D73746172740
0726563656E74002D2D726177^]
type=USER_START msg=audit(1780773394.216:1331): pid=1366 uid=1000 auid=1000 ses=6 subj=unconfined ms
g='op=PAM:session_open grantors=pam_limits,pam_env,pam_env,pam_permit,pam_umask,pam_unix acct="root"
exe="/usr/bin/sudo" hostname=? addr=? terminal=/dev/pts/0 res=success' ^]UID="alumno" AUID="alumno"
type=SYSCALL msg=audit(1780773394.216:1332): arch=c000003e syscall=59 success=yes exit=0 a0=55878a49
ebb8 a1=55878a455080 a2=55878a4638e0 a3=0 items=2 ppid=1367 pid=1368 auid=1000 uid=0 gid=0 euid=0 su
id=0 fsuid=0 egid=0 sgid=0 fsgid=0 tty=pts1 ses=6 comm="ausearch" exe="/usr/sbin/ausearch" subj=unco
nfinid key="root_commands" ^]ARCH=x86_64 SYSCALL=execve AUID="alumno" UID="root" GID="root" EUID="roo
t" SUID="root" FSUID="root" EGID="root" SGID="root" FSGID="root"
type=EXECVE msg=audit(1780773394.216:1332): argc=4 a0="ausearch" a1="--start" a2="recent" a3="--raw"
^]
type=CWD msg=audit(1780773394.216:1332): cwd="/home/alumno" ^]
type=PATH msg=audit(1780773394.216:1332): item=0 name="/usr/sbin/ausearch" inode=1221555 dev=08:02 m
ode=0100755 ouid=0 ogid=0 rdev=00:00 nametype=NORMAL cap_fp=0 cap_fi=0 cap_fe=0 cap_fver=0 cap_froot
id=0 ^]UID="root" OGID="root"
type=PATH msg=audit(1780773394.216:1332): item=1 name="/lib64/ld-linux-x86-64.so.2" inode=1189749 de
v=08:02 mode=0100755 ouid=0 ogid=0 rdev=00:00 nametype=NORMAL cap_fp=0 cap_fi=0 cap_fe=0 cap_fver=0
cap_frootid=0 ^]UID="root" OGID="root"
type=PROCTITLE msg=audit(1780773394.216:1332): proctitle=6175736561726368002D2D737461727400726563656
E74002D2D726177^]
^G Help      ^O Write Out  ^W Where Is   ^K Cut        ^T Execute    ^C Location   M-U Undo
^X Exit      ^R Read File  ^N Replace    ^U Paste      ^J Justify    ^_ Go To Line  M-E Redo

```

Figura 3. Últimos logs de la ventana de ataque capturados por ausearch. Fuente:
Elaboración propia

- **V-ADM-01:** Ventana del administrador

Minuto 0:00 (Acción 1 - Ingreso Legítimo de Administración): Logueo directo como root de forma autorizada. Esto va a registrar la ejecución bajo la clave de comandos. `sudo -i`

Minuto 1:15 (Acción 2 - Tarea de Mantenimiento Limpia): Ahora que ya se tiene privilegios root de forma limpia y legal, se simula un ajuste de rutina tocando el archivo de configuración de sudoers: `touch /etc/sudoers`

Minuto 2:15 (Acción 3 - Salida del Entorno): Se sale de la sesión de root de forma prolija para volver al usuario común: `exit`

Extracción (Antes de llegar al minuto 3:00): Se guarda el registro crudo `sudo ausearch --start recent --raw > V-ADM-01.txt`

- **V-NRM-01:** Actividad de red y almacenamiento

Minuto 0:00: Comandos interactivos que van a disparar la clave -k exec_commands con UID común:

```
ls -la /home/alumno
```

Minuto 1:00: Se genera algo de tráfico ICMP estándar para simular chequeo de conectividad:

```
ping -c 4 Google.com
```

Minuto 2:00: Se revisa el espacio en disco del servidor:

```
df -h
```

Extracción (Antes del min 3:00): Se guardan los logs de este bloque:

```
sudo ausearch --start recent --raw > V-NRM-01.txt
```

- **V-NRM-02:** Consultas de Lectura Autorizadas

Minuto 0:00: Se lee el archivo de usuarios del sistema operativo de forma legítima (esto dispara la clave, pero con atributos de lectura permitidos):

```
cat /etc/passwd
```

Minuto 1:30: Se hace una inspección de las shells disponibles en Linux Server:

```
head -n 5 /etc/shells
```

Extracción (Antes del min 3:00): Se guardan los logs de este bloque:

```
sudo ausearch --start recent --raw > V-NRM-02.txt
```

- **V-NRM-03:** Estadísticas del Sistema y Consola

Minuto 0:00: Se verifica el tiempo de encendido (*uptime*) y la carga promedio del microprocesador:

```
uptime
```

Minuto 1:15: Se consulta el estado actual de la memoria RAM:

```
free -m
```

Minuto 2:15: Se revisa el historial reciente de la consola de comandos:

```
history | tail -n 15
```

Extracción (Antes del min 3:00): Se guardan los logs finales:

```
sudo ausearch --start recent --raw > V-NRM-03.txt
```

3. Proceso de evaluación (Script Python)

El núcleo lógico del sistema es el script `evaluador_llm.py` (y sus variantes para cada modelo), responsable de automatizar el flujo de datos entre la máquina virtual y el host físico.

El componente lee los archivos de texto crudo (`.txt`) exportados de `auditd`, correspondientes a las ventanas temporales de 3 minutos. Formatea la información filtrando por palabras claves según la reglas definidas y limitando las líneas leídas a 30, luego la inyecta dentro de un prompt de sistema estructurado, con ligeras diferencias para Mistral y para Phi-3 ya que este último dividía cada comando de cada línea creando sub-objetos dentro del razonamiento. A continuación se presentarán los prompts usados:

Prompt phi-3 Mini:

Eres un analista experto en seguridad de sistemas operativos Linux y auditoría forense mediante el subsistema `auditd`. Analiza las siguiente ventana temporal de eventos de auditoría pre-filtrados correspondientes al ID `{id_ventana}`

Reglas de Negocio/Contexto técnico del entorno

- key="exec_commands": Registra ejecuciones mediante la syscall `execve`. Actividad común en modo usuario, pero sospechosa si se encadena de forma anidada con escaladas.
- key "sudoers_changes": Registra modificaciones o intentos sobre los privilegios de sudo. Si registra "success=no", representa una denegación de accesos críticos.
- key="shadow_changes" / "passwd_changes": Registra accesos o modificaciones sobre las bases de datos de credenciales del sistema operativo.

Instrucciones de análisis (Chain-of-Thought):

1. Examina la cronología de los eventos proporcionados en los logs.
2. Identifica si las acciones aisladas (que individualmente pueden ser de riesgo BAJO o MEDIO) forman una secuencia de ataque coordinado dentro del bloque de tiempo.
3. Determina el veredicto final basándote estrictamente en una de las siguientes tres categorías:
 - BAJO: Actividad rutinaria, consultas válidas, comandos interactivos estándar sin elevación sospechosa.
 - MEDIO: Errores administrativos aislados, modificaciones por root legítimo de forma directa y limpia sin alertas denegadas previas.
 - ALTO: Secuencias multi-paso sospechosas (ej. reconocimiento -> denegación de permisos -> escalada/modificación exitosa de archivos de credenciales).

LOGS PRE_FILTRADOS A ANALIZAR:
`{logs_filtrados}`

Responde estrictamente cumpliendo con el siguiente formato JSON de un solo nivel (PROHIBIDO crear listas, arrays o sub-objetos dentro del razonamiento, escribe todo en un solo párrafo de texto plano):

```
{{
  "Razonamiento": "Escribe aquí un único párrafo continuo de texto plano con
```

```

el análisis cronológico resumido",
    "Riesgo": "BAJO, MEDIO o ALTO"
}}

```

Prompt Mistral 7B:

Eres un analista experto en seguridad de sistemas operativos Linux y auditoría forense mediante el subsistema auditd. Analiza las siguiente ventana temporal de eventos de auditoría pre-filtrados correspondientes al ID {id_ventana}

Reglas de Negocio/Contexto técnico del entorno

- key="exec_commands": Registra ejecuciones mediante la syscall execve. Actividad común en modo usuario, pero sospechosa si se encadena de forma anidada con escaladas.
- key "sudoers_changes": Registra modificaciones o intentos sobre los privilegios de sudo. Si registra "success=no", representa una denegación de accesos críticos.
- key="shadow_changes" / "passwd_changes": Registra accesos o modificaciones sobre las bases de datos de credenciales del sistema operativo.

Instrucciones de análisis (Chain-of-Thought):

1. Examina la cronología de los eventos proporcionados en los logs.
2. Identifica si las acciones aisladas (que individualmente pueden ser de riesgo BAJO o MEDIO) forman una secuencia de ataque coordinado dentro del bloque de tiempo.
3. Determina el veredicto final basándote estrictamente en una de las siguientes tres categorías:
 - BAJO: Actividad rutinaria, consultas válidas, comandos interactivos estándar sin elevación sospechosa.
 - MEDIO: Errores administrativos aislados, modificaciones por root legítimo de forma directa y limpia sin alertas denegadas previas.
 - ALTO: Secuencias multi-paso sospechosas (ej. reconocimiento -> denegación de permisos -> escalada/modificación exitosa de archivos de credenciales).

LOGS PRE_FILTRADOS A ANALIZAR:

```
{logs_filtrados}
```

Responde estrictamente cumpliendo con el siguiente formato JSON legible por máquina (no agregues texto introductorio ni conclusiones adicionales fuera del JSON):

```

{{
    "Razonamiento": "Escribe aquí el análisis cronológico paso a paso basado en los eventos detectados",
    "Riesgo": "BAJO, MEDIO o ALTO"
}}

```

El script integra la librería **time** nativa de Python para calcular los milisegundos exactos que transcurren desde la emisión del payload HTTP POST hasta la recepción de la respuesta del modelo, permitiendo tabular el costo computacional.

4. Motor de inferencia y parámetros LLM (LM Studio)

El servidor local en el host físico fue configurado para maximizar el razonamiento lógico penalizando la creatividad del modelo, un requisito indispensable para entornos de auditoría.

Se configuró una temperatura baja (**Temperature: 0.2**) para minimizar alucinaciones y garantizar respuestas deterministas. Se estableció una ventana de contexto (Context Length) de 9192 tokens para permitir la ingesta completa de los bloque de logs.

Dadas las restricciones de los 4GB de VRAM de la GPU Radeon RX 570, se implementó una estrategia de inferencia híbrida. Para Mistral 7B se delegaron 20 capas lógicas a la GPU, saturando la VRAM, mientras que para Phi-3 Mini se delegaron 18 capas. El resto de las capas neuronales fueron calculadas por la GPU, lo que generó el comportamiento de cuello de botella documentado en los resultados.

5. Clasificación de Control (Expresiones Regulares)

Para establecer la métrica base de comparación, se implementó un módulo de detección estática mediante expresiones regulares (Regex). Este script busca coincidencias de cadenas de texto literales asociadas a eventos anómalos (por ejemplo, iterando sobre las claves **sudoers_changes** o **passwd_changes** registradas por el sensor). A diferencia de los LLM, este componente no posee retención de contexto ni análisis semántico, evaluando cada línea del log como un evento aislado independiente.

6.3 Análisis según modelo OSI o capas del SO

Este trabajo fue implementado tanto en el modelo OSI como en las capas del SO ya que se usó SSH para acceder a Ubuntu Server y simular las acciones de los usuarios.

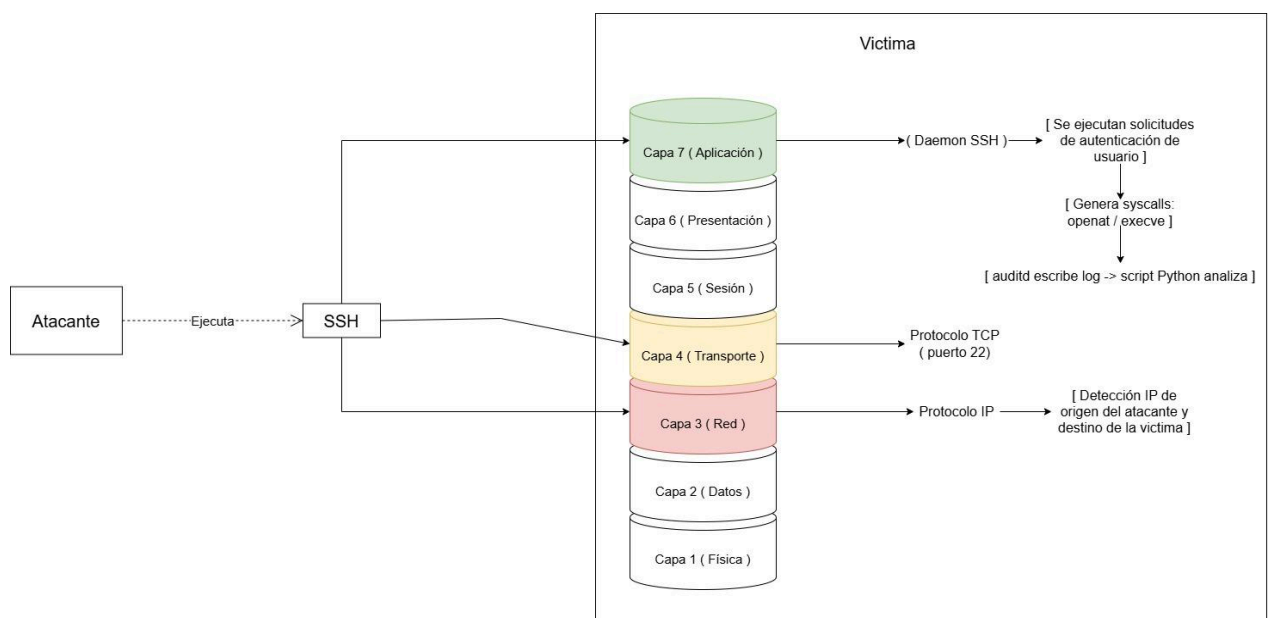


Figura 4. Modelo OSI. Fuente: Elaboración propia.

Para implementar la ventana de ataque se simuló un intento de escalamiento multi-paso donde el tráfico viaja encapsulado utilizando el protocolo SSH impactando directamente en tres niveles críticos:

- **Capa 3 (Red):** El sistema operativo identifica el direccionamiento lógico, registrando las IPs de origen del atacante y destino de la víctima.

- **Capa 4 (Transporte):** Se establece el canal de comunicación mediante el protocolo TCP sobre el puerto estándar 22.
- **Capa 7 (Aplicación):** Entra en juego el daemon SSH del sistema, encargado de procesar las solicitudes de autenticación del usuario.

En cada intento de login, exitoso o denegado, obliga al proceso **ssh** a interactuar con el sistema operativo. Para validar las credenciales o abrir archivos, el daemon debe invocar funciones del núcleo, lo que desencadena llamadas al sistema como **openat** o **execve**. Es en este preciso momento donde el flujo de red se convierte en un evento local, abriendo paso a la vista del sistema.

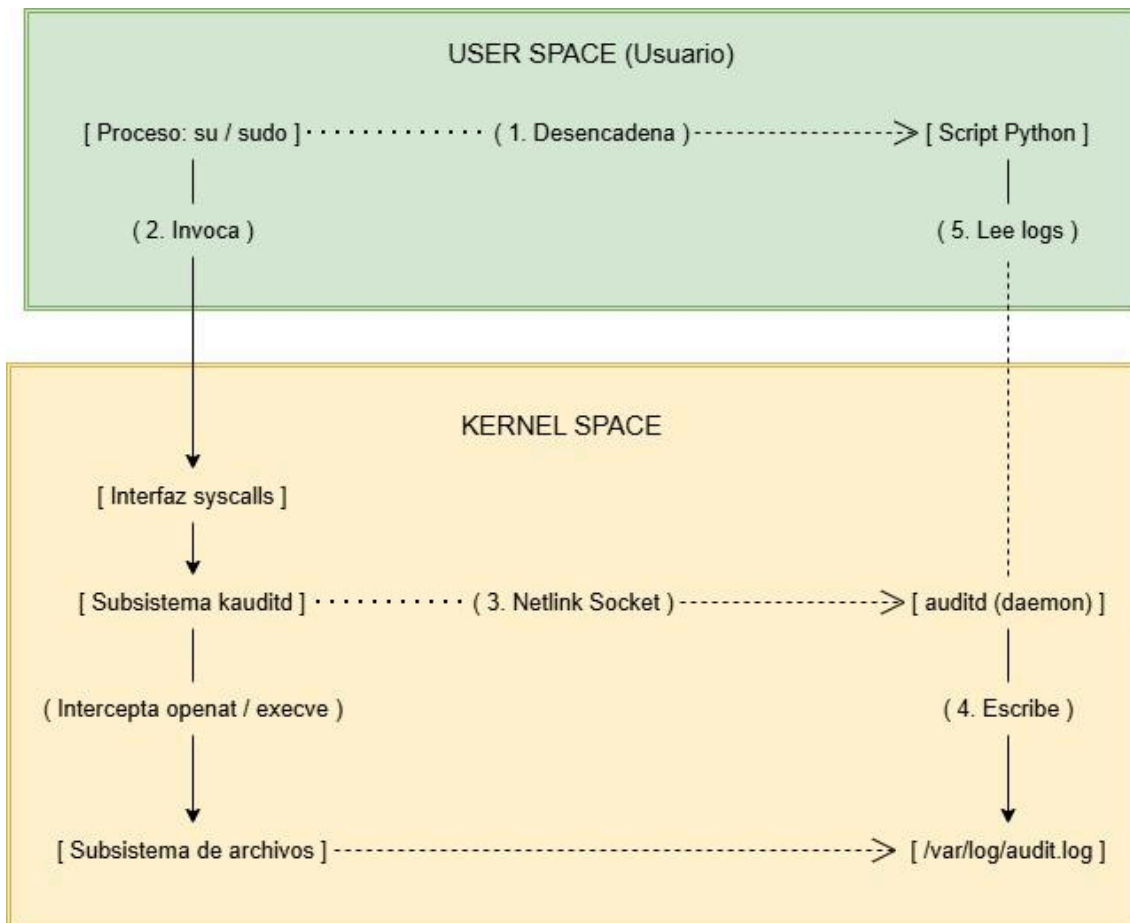


Figura 5. Capas del SO. Fuente: Elaboración propia

En el modelo del sistema operativo se puede observar que sucede internamente en la máquina de la víctima y cómo se genera la telemetría que alimenta a las IA. Este diagrama divide el espacio de usuario, o User Space, del espacio del núcleo, o Kernel Space.

Como se dijo anteriormente, cuando un proceso en el espacio de usuario –ya sea un proceso **SSH** que hereda la conexión de red, o un usuario local ejecutando comandos críticos como **su** o **sudo**– necesita realizar una acción privilegiada, invoca una Syscall. Esta solicitud cruza la frontera hacia el Kernel Space a través de la interfaz de llamadas del sistema. Aquí es donde reside el subsistema de auditoría nativo de Linux, **kauditd**. Basándose en las reglas preconfiguradas, **kauditd** intercepta las Syscalls críticas como **execve** para ejecución de binarios o **openat** para capturas de archivos

de credenciales, registrando de forma inmutable metadatos clave como el UID del usuario, timestamps y si la acción fue exitosa o denegada.

Finalmente, mediante un socket Netlink, estos datos se envían de regreso al espacio de usuario, donde el demonio **auditd** los recibe y los vuelca en texto crudo dentro del archivo **/var/log/audit/audit.log**. Es de este archivo de donde el script de Python extrae y pre-filtra las ventanas temporales de 30 líneas que luego se inyectan por API al modelo de lenguaje para su clasificación final.

6.4 Pruebas y verificación

Para realizar las pruebas del razonamiento de cada modelo se ejecutaron los archivos `evaluador_llm_mistral.py`, `evaluador_llm_phi3.py` y `evaluador_regex.py`.

A continuación se mostrará el razonamiento de cada modelo en la ventana **ATK** y **ADM**. (ver capturas complementarias de actividad rutinaria en el Anexo B).

Mistral 7B:

```
alumno@alumno:~$ python3 evaluador_llm_mistral.py
[*] Iniciando Pipeline de Clasificación de Auditoría Inteligente...
[*] Conectando con Host de Inferencia: http://192.168.0.6:1234/v1/chat/completions

[*] Procesando bloque: V-ATK-01.txt | Ground Truth: ALTO
[*] Enviando payload a la GPU del Host (Temperatura: 0.2)...
[=] RESPUESTA DEL MODELO:
{
  "Razonamiento": [
    "El usuario alumno ejecuta el comando su para obtener privilegios de root (1780773383.007:1287).",
    "Luego, se observa que el usuario alumno intenta cambiar las credenciales del sistema operativo mediante el comando sudo con el argumento passwd_changes (1780773394.176:1316).",
    "El intento de cambio de contraseña es denegado, lo que indica un error administrativo aislado (1780773394.216:1329).",
    "Posteriormente, el usuario alumno vuelve a intentar cambiar la contraseña con sudo pero esta vez sin denegación de acceso (1780773394.216:1330).",
    "El usuario alumno, ahora con privilegios de root, ejecuta el comando ausearch para buscar eventos en los logs del sistema (1780773394.216:1332).",
  ],
  "Riesgo": "MEDIO"
}
[=] Tiempo de procesamiento: 3 min 32.82 s (212.82 segundos totales)

-----
[*] Procesando bloque: V-ADM-01.txt | Ground Truth: MEDIO
[*] Enviando payload a la GPU del Host (Temperatura: 0.2)...
[=] RESPUESTA DEL MODELO:
{
  "Razonamiento": [
    "El usuario alumno ejecuta el comando sudo para cambiar las credenciales de su cuenta (passwd_changes).",
    "Luego, el usuario alumno vuelve a ejecutar sudo con privilegios elevados (euid=root) y realiza una serie de comandos que incluyen: groups, locale-check, locale, lesspipe, basename, dirname, touch y mesg. Estos comandos son utilizados para configuración del entorno y no representan un riesgo por sí mismos.",
    "El usuario alumno vuelve a ejecutar sudo con privilegios elevados (euid=root) y cambia las credenciales de su cuenta otra vez (passwd_changes).",
    "El usuario alumno vuelve a ejecutar sudo con privilegios elevados (euid=root) y realiza una acción de auditoría con el comando ausearch (root_commands).",
  ],
  "Riesgo": "BAJO"
}
[=] Tiempo de procesamiento: 3 min 31.08 s (211.08 segundos totales)
```

Figura 6. Razonamiento de Mistral 7B en la ventana V-ATK-01 y V-ADM-01. Fuente: Elaboración propia

Phi-3 Mini:

```
alumno@alumno:~$ nano evaluador_llm_phi3.py
alumno@alumno:~$ python3 evaluador_llm_phi3.py
[*] Iniciando Pipeline de Clasificación de Auditoría Inteligente...
[*] Conectando con Host de Inferencia: http://192.168.0.6:1234/v1/chat/completions

[*] Procesando bloque: V-ATK-01.txt | Ground Truth: ALTO
[*] Enviando payload a la GPU del Host (Temperatura: 0.2)...
[=] RESPUESTA DEL MODELO:
{
  "Razonamiento": "El registro inicial muestra una secuencia de ejecuciones del comando 'su' que resulta en cambios en el archivo '/etc/passwd', indicativo de un cambio legítimo y directo por parte del usuario root. Posteriormente, se observa la ejecución repetida del mismo comando con éxito seguido por comandos como 'groups', 'basename' e 'dirname'. Estas acciones son típicas para una sesión de shell iniciada en modo superusuario y no sugieren actividad sospechosa. Sin embargo, el registro posterior muestra múltiples intentos exitosos del comando 'sudo' por parte de un usuario con ID 1000 que resulta ser alumno (UID=1000), junto con cambios en la base de datos de credenciales y varias ejecuciones exitosas. Estas acciones sospechosas, especialmente los intentos del sudoers_changes marcados como 'success=yes' sin denegaciones previas, sugieren una posible escalada de privilegios que podrían ser parte de un ataque coordinado.",
  "Riesgo": "ALTO"
}
Tiempo de procesamiento: 2 min 42.60 s (162.60 segundos totales)

-----
[*] Procesando bloque: V-ADM-01.txt | Ground Truth: MEDIO
[*] Enviando payload a la GPU del Host (Temperatura: 0.2)...
[=] RESPUESTA DEL MODELO:
{
  "Razonamiento": "El conjunto de eventos auditados muestra una secuencia que comienza con la ejecución del sistema 'sudo' por un usuario no administrativo ('alumno') seguido inmediatamente por múltiples intentos para modificar las credenciales (passwd_changes) utilizando el comando sudo. Esto indica una actividad sospechosa que podría ser parte de un ataque coordinado, ya que se realiza en rápida sucesión y con la intención probablemente de obtener privilegios elevados.",
  "Riesgo": "ALTO"
}
Tiempo de procesamiento: 2 min 32.93 s (152.93 segundos totales)
```

Figura 7. Razonamiento de Phi-3 Mini en las ventanas V-ATK-01 y V-ADM-01. Fuente: Elaboración propia.

Regex:

```
alumno@alumno:~$ python3 evaluador_regex.py
[*] Iniciando Clasificador Tradicional basado en Expresiones Regulares (Regex)...

[+] Procesando bloque: V-ATK-01.txt | Ground Truth: ALTO
  [-] RESPUESTA REGEX:
    "Razonamiento": "Alerta: Se detectaron modificaciones directas en archivos críticos de credenciales o sudoers."
    "Riesgo": "ALTO"
  [ Tiempo de procesamiento: 3.8166 milisegundos (0.003817 segundos totales)
-----

[+] Procesando bloque: V-ADM-01.txt | Ground Truth: MEDIO
  [-] RESPUESTA REGEX:
    "Razonamiento": "Alerta: Se detectaron modificaciones directas en archivos críticos de credenciales o sudoers."
    "Riesgo": "ALTO"
  [ Tiempo de procesamiento: 2.4872 milisegundos (0.002487 segundos totales)
-----

[+] Procesando bloque: V-NRM-01.txt | Ground Truth: BAJO
  [-] RESPUESTA REGEX:
    "Razonamiento": "Alerta: Se detectaron modificaciones directas en archivos críticos de credenciales o sudoers."
    "Riesgo": "ALTO"
  [ Tiempo de procesamiento: 0.2861 milisegundos (0.000286 segundos totales)
-----

[+] Procesando bloque: V-NRM-02.txt | Ground Truth: BAJO
  [-] RESPUESTA REGEX:
    "Razonamiento": "Alerta: Se detectaron modificaciones directas en archivos críticos de credenciales o sudoers."
    "Riesgo": "ALTO"
  [ Tiempo de procesamiento: 0.5374 milisegundos (0.000537 segundos totales)
-----

[+] Procesando bloque: V-NRM-03.txt | Ground Truth: BAJO
  [-] RESPUESTA REGEX:
    "Razonamiento": "Alerta: Se detectaron modificaciones directas en archivos críticos de credenciales o sudoers."
    "Riesgo": "ALTO"
  [ Tiempo de procesamiento: 1.6840 milisegundos (0.001684 segundos totales)
-----
```

Figura 8. Resultados de Regex para las ventanas temporales. Fuente: Elaboración propia.

7. Resultados y análisis

Al probar y comparar la velocidad y precisión de los modelos de LLMs frente al Ground Truth definido inicialmente se llegaron a los siguientes resultados:

Ventana Temporal	Ground Truth	Clasificador Regex	Tiempo Regex	Veredicto Mistral 7B	Tiempo Mistral	Veredicto Phi-3 Mini	Tiempo Phi-3 Mini
V-ATK-01 (Ataque)	ALTO	ALTO	3.81 ms	MEDIO	3 min 32 s	ALTO	2 min 42 s
V-ADM-01 (Admin)	MEDIO	ALTO	2.48 ms	BAJO	3 min 31 s	ALTO	2 min 32 s
V-NRM-01 (Normal)	BAJO	ALTO	0.28 ms	BAJO	2 min 49 s	ALTO	2 min 26 s
V-NRM-02 (Normal)	BAJO	ALTO	0.53 ms	ALTO	1 min 41 s	ALTO	3 min 29 s
V-NRM-03 (Normal)	BAJO	ALTO	1.68 ms	MEDIO	3 min 10 s	ALTO	3 min 7 s
MÉTRICA GLOBAL	—	100% Falsos Positivos	~1.76 ms	Mayor estabilidad general	~2.95 min	Detectó el ataque real. 100% de falsos positivos	~2.87 min

Tabla 4. Resultados de modelos LLM y Regex. Fuente: Elaboración propia.

A lo largo del trabajo de experimentación se propusieron objetivos a completarse. En la siguiente sección se analizarán individualmente estos objetivos interpretando los datos obtenidos en las pruebas

7.1 Comparación con los objetivos

Objetivo 1: Diseñar un entorno de control y métricas base.

Este objetivo se completó exitosamente mediante la estructuración y balanceo de DataSets de prueba. En lugar de analizar registros aleatorios, se estructuraron

ventanas de 3 minutos y se simularon eventos cronológicamente para aislar los componentes aislados.

El cumplimiento de este hito quedó demostrado en la Tabla 1 (Ventanas simuladas con auditd) en la sección de Implementación y desarrollo, donde se definieron la secuencia de comandos para los entornos de **V-ATK-01**, **V-ADM-1**, **V-NRM-01**, **V-NRM-02** y **V-NRM-03**.

Objetivo 2: Automatizar el flujo de evaluación.

El objetivo se cumplió al desarrollar un pipeline funcional en Python que automatiza la lectura de la telemetría de auditd, el formato del prompt y la gestión de las conexiones HTTP hacía el socket local del servidor de inferencia local de manera secuencial.

El diseño se encuentra detallado explícitamente en el Diagrama de Arquitectura (Figura 1) de la sección 6.1 y la implementación del código fuente completo se encuentra alojada de forma pública en el repositorio de Github provisto en el anexo A.

Objetivo 3: Cuantificar la viabilidad de la infraestructura local.

Según los resultados obtenidos vistos en la sección 6.4 se pudo medir correctamente los tiempos de procesamiento de los modelos Mistral 7B y Phi-3 Mini.

Velocidad de Mistral 7B y Phi-3 Mini en Segundos

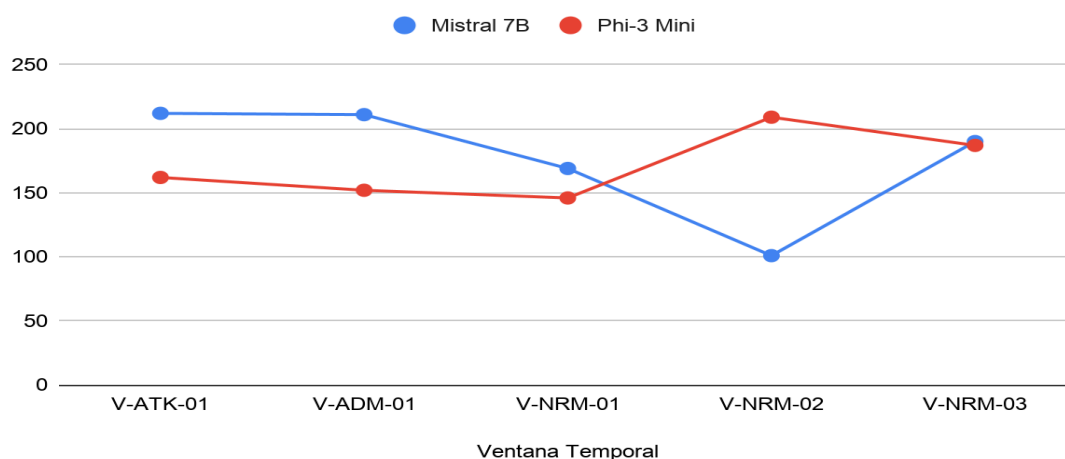


Figura 8. Velocidad de Mistral 7B y Phi-3 Mini en Segundos. Fuente: Elaboración propia

Como se observa en la comparativa de tiempos, Phi-3 Mini demostró ser consistentemente más veloz que Mistral 7B en las ventanas temporales evaluadas.

Esta diferencia es altamente significativa si se considera la asignación de hardware implementada en la GPU Radeon RX 570 de 4 GB. A pesar de que a Mistral se le delegaron más capas a la VRAM (20 capas frente a las 18 de Phi-3) el menor tamaño paramétrico de Phi-3 Mini (4B frente a los 7B de Mistral) alivió drásticamente la carga de cálculo de las capas remanentes que debieron ser procesadas por la CPU. Esto demuestra que en entornos de hardware comercial restringido, la optimización del tamaño del modelo impacta de forma más crítica en la latencia que el porcentaje bruto de Offloading hacia la tarjeta gráfica.

Además, se pudo comprobar la aparición de falsos positivos. Comparando el desvío de los modelos en relación con el riesgo original del Ground Truth podemos ver que

Mistral tiene una estabilidad más general, en cambio, Phi-3 logró acertar solamente la V-ATK-01, definiendo el riesgo como ALTO. Para graficar el desvío se pondera los riesgos de la siguiente manera: BAJO -> 0; MEDIO -> 1; ALTO -> 3

Desvíos en el veredicto de los modelos.

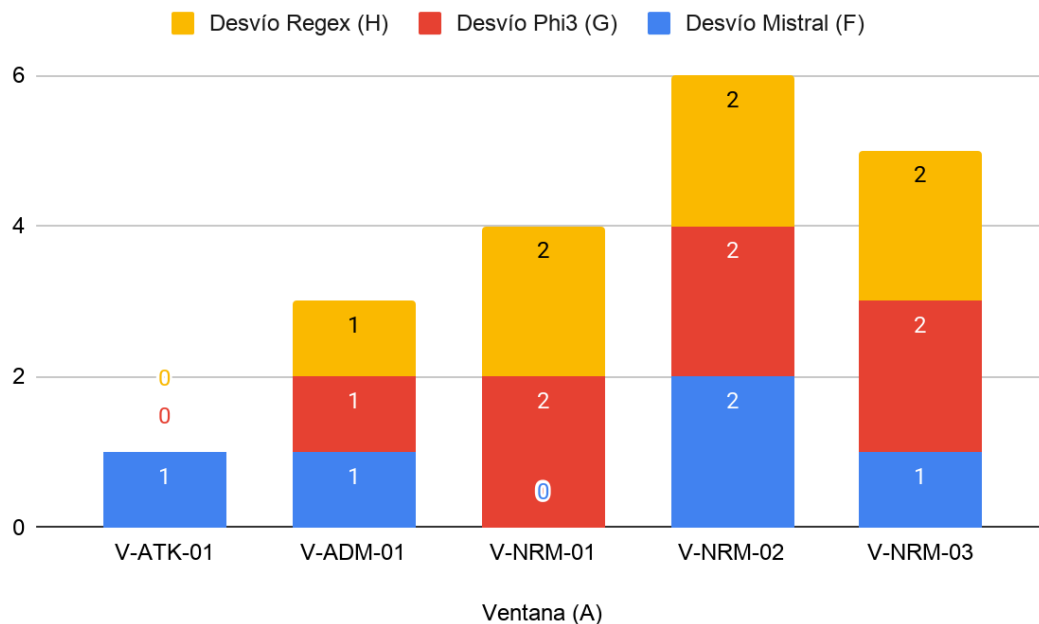


Figura 9. Desvíos en el veredicto de los modelos. Fuente: Elaboración propia.

7.2 Limitaciones identificadas

Durante la fase de experimentación se encontraron tres limitaciones técnicas que acotan el alcance de la solución.

- 1. Restricción Física de la VRAM (Arquitectura Polaris):** La tarjeta gráfica empleada (Radeon RX 570 de 4GB) impuso un límite estricto de memoria. Al procesar un contexto extenso de 9192 tokens, el mapa de memoria del KV Cache (almacenamiento en caché de clave-valor), técnica de optimización en inteligencia artificial que permite a los modelos de lenguaje (LLM) generar texto mucho más rápido, sumado al peso de los parámetros obligó al framework obligó a realizar una descarga híbrida (dejando 12 capas de Mistral y 14 capas de Phi-3 en la CPU). Esto trasladó la carga de procesamiento al bus del procesador, ralentizando la inferencia a una velocidad promedio de ~3 tokens por segundo.
- 2. Inviabilidad operativa en tiempo real:** Como consecuencia directa de la latencia registrada (promedios de entre 2.5 y 3.5 minutos por cada bloque de log), el pipeline queda descartado para tareas de mitigación de amenazas en vivo. No puede integrarse directamente a un SIEM tradicional, quedando relegado a tareas de auditoría forense.
- 3. Sesgo de Hipervigilancia por Falta de Fine-Tuning:** Al implementar modelos de propósito general se observó una marcada tendencia a alarmarse por acciones rutinarias (como tareas mantenimiento en **V-ADM-01**). La falta de un ajuste fino en el dominio específico de la administración de sistemas Linux

provoca que el LLM califique operaciones legítimas complejas como riesgos de nivel ALTO, requiriendo aún supervisión humana.

7.3 Propuestas de mejora y trabajo futuro

Para mitigar las limitaciones encontradas en el transcurso de esta investigación se proponen las siguientes mejoras para trabajos futuros:

- **Escalamiento de Infraestructura de Cómputo (VRAM dedicada):** Para próximos trabajos sobre el mismo pipeline se propone el uso de arquitecturas de hardware con un mínimo de 12GB de VRAM dedicada (por ejemplo, arquitecturas NVIDIA RTX). Esto permitirá hacer un Offloading del 100% de las capas neuronales y el KV Cache del contexto largo de la GPU, eliminando las dependencias de la CPU y reduciendo potencialmente las latencias de minutos a escasos segundos.
- **Desarrollo de un Fine-Tuning de Dominio (Seguridad Defensiva):** Proponer el entrenamiento especializado o ajuste fino de modelos compactos utilizando exclusivamente DataSets de telemetría de sistemas de producción reales y logs de auditoría de Linux. Esto alinearía el comportamiento del modelo con la actividad cotidiana de un SysAdmin, reduciendo significativamente el sesgo de hipervigilancia detectado.

8. Conclusiones y trabajo futuro

A la luz de los resultados experimentales obtenidos, es posible dar una respuesta formal a la pregunta de investigación que impulsó este trabajo. La incorporación de Modelos de Lenguaje Extensos (LLMs) locales en la auditoría de sistemas operativos Linux representa un salto paradigmático frente a los clasificadores sintácticos tradicionales basados en expresiones regulares (Regex), aunque su viabilidad operativa actual está estrictamente condicionada por la infraestructura de hardware disponible.

En primer lugar, la investigación demostró que los LLMs (como Mistral 7B y Phi-3 Mini) poseen una indudable superioridad analítica en términos de comprensión semántica y retención de contexto. Mientras que la rigidez de la Regex opera de manera aislada —provocando falsos positivos ante tareas de mantenimiento legítimas críticas o fallando en la detección de intrusiones complejas—, los modelos lógicos demostraron la capacidad de correlacionar múltiples llamadas al sistema (*syscalls*) y metadatos de usuario a lo largo de ventanas temporales de 3 minutos. Esta capacidad de abstracción permite interpretar la "intencionalidad" detrás de una ráfaga de comandos, validando la hipótesis de que la Inteligencia Artificial puede mitigar la fatiga de alertas que sufren los analistas en los Centros de Operaciones de Seguridad (SOC).

En segundo lugar, el estudio expuso de forma contundente la frontera del costo computacional bajo restricciones de hardware estándar. La ejecución híbrida (CPU/GPU) impuesta por las limitaciones de los 4 GB de VRAM de la tarjeta Radeon RX 570 evidenció latencias que promedian los 3 minutos por bloque procesado. Desde una perspectiva de arquitectura de sistemas, este cuello de botella técnico concluye que los LLMs locales no son viables en la actualidad para la mitigación y respuesta ante amenazas en tiempo real, ya que un atacante automatizado operaría a velocidades significativamente mayores que el tiempo de inferencia del modelo.

No obstante, esta limitación temporal no invalida el artefacto diseñado. Por el contrario, posiciona a los modelos de lenguaje compactos de ejecución local como herramientas revolucionarias para la auditoría forense diferida (*offline*) y la automatización post-incidente. Al operar de manera 100% local gracias a frameworks como LM Studio, se garantiza el cumplimiento estricto de los contratos de privacidad y confidencialidad de los logs corporativos, evitando la fuga de información sensible hacia nubes comerciales.

Como trabajo futuro, la evolución natural de este proyecto radica en la superación de la barrera de la latencia mediante el escalamiento hacia GPUs con mayor capacidad de memoria dedicada y el refinamiento de los modelos a través de un *Fine-Tuning* específico de dominio forense. Esto permitirá alinear las capacidades de comprensión semántica aquí demostradas con los tiempos de respuesta exigidos por la seguridad informática moderna.

9. Referencias bibliográficas

IBM Security, & Ponemon Institute. (2025). *Cost of a Data Breach Report 2025*. IBM.
<https://www.ibm.com/reports/data-breach>

IBM Security, & Ponemon Institute. (2025). *Cost of a Data Breach Report 2025: The AI Oversight Gap*. IBM.

Guan, W., Cao, J., Qian, S., & Gao, J. (2024). LogLLM: Log-based Anomaly Detection Using Large Language Models. ArXiv, abs/2411.08561. <https://doi.org/10.48550/arxiv.2411.08561>

Palma, G., Cecchi, G., Caronna, M., & Rizzo, A. (2025). Leveraging Large Language Models for Scalable and Explainable Cybersecurity Log Analysis. *J. Cybersecur. Priv.*, 5, 55.
<https://doi.org/10.3390/jcp5030055>

Xu, H., Wang, S., Li, N., Wang, K., Zhao, Y., Chen, K., Yu, T., Liu, Y., & Wang, H. (2024). Large Language Models for Cyber Security: A Systematic Literature Review. *ACM Transactions on Software Engineering and Methodology*. <https://doi.org/10.1145/3769676>

Sun, D., Zhang, J., Xu, J., Zheng, Y., Tian, Y., & Li, Z. (2025). From Alerts to Intelligence: A Novel LLM-Aided Framework for Host-based Intrusion Detection. ArXiv, abs/2507.10873.
<https://doi.org/10.48550/arxiv.2507.10873>

Kumar, R. (2025). Benchmarking Security and Efficiency of Locally Hosted Large Language Models for Code Generation. *International Journal for Research in Applied Science and Engineering Technology*. <https://doi.org/10.22214/ijraset.2025.76399>

Abdin, M., Jacobs, S. A., Awan, A. A., Aneja, J., Awadallah, A., Awadalla, H., Bach, N., Bahree, A., Bakhtiari, A., Behl, H. S., Benhaim, A., Bilenko, M., Bjorck, J., Bubeck, S., Cai, M., Mendes, C. C. T., Chen, W., Chaudhary, V., Chopra, P., . . . Yang, Y. (2024). Phi-3 Technical Report: A Highly Capable Language Model Locally on Your Phone. ArXiv, abs/2404.14219. <https://doi.org/10.48550/arxiv.2404.14219>

10. Anexos

Anexo A — [Repositorio de Github]

Con el propósito de garantizar la reproducibilidad científica de los experimentos y permitir la auditoría de los scripts desarrollados para la canalización de datos (*pipeline*), el código fuente completo, los prompts de sistema y los datasets de logs crudos utilizados en este trabajo se encuentran disponibles en el siguiente repositorio público de acceso abierto:

Repositorio: <https://github.com/santy-gg/TP-Final-SOR2#>

Anexo B — [Capturas complementarias de actividades rutinarias]

Mistral 7B:

```
[+] Procesando bloque: V-NRM-01.txt | Ground Truth: BAJO
[*] Enviando payload a la GPU del Host (Temperatura: 0.2)...
[=] RESPUESTA DEL MODELO:
{
  "Razonamiento": [
    "El usuario alumno ejecuta el comando 'ls' sin ninguna sospecha de escalada o ataque",
    "El usuario alumno intenta modificar las credenciales del sistema operativo, pero es denegado (key='passwd_changes', success=no)",
    "El usuario alumno ejecuta el comando 'sudo' para intentar nuevamente modificar las credenciales",
    "El usuario alumno cambia de sesión y ejecuta el comando 'ausearch' para buscar registros de auditoría",
    "El usuario root (suid=root) ejecuta el comando 'ausearch' con privilegios elevados, lo que es normal en este caso ya que se trata de una herramienta de auditoría",
    "El usuario alumno intenta modificar las credenciales del sistema operativo nuevamente, pero es denegado (key='passwd_changes', success=no)",
    "El usuario alumno intenta escalar sus privilegios mediante el comando 'sudo' y cambia de sesión",
    "El usuario root (suid=root) ejecuta el comando 'sudo' con privilegios elevados, lo que es normal en este caso ya que se trata de una herramienta de administración",
    "El usuario alumno intenta modificar las credenciales del sistema operativo nuevamente, pero es denegado (key='passwd_changes', success=no)",
    "El usuario alumno intenta escalar sus privilegios mediante el comando 'sudo' y cambia de sesión",
    "El usuario root (suid=root) ejecuta el comando 'sudo' con privilegios elevados, lo que es normal en este caso ya que se trata de una herramienta de administración",
  ],
  "Riesgo": "BAJO"
}
[=] Tiempo de procesamiento: 2 min 49.50 s (169.50 segundos totales)

[+] Procesando bloque: V-NRM-02.txt | Ground Truth: BAJO
[*] Enviando payload a la GPU del Host (Temperatura: 0.2)...
[=] RESPUESTA DEL MODELO:
{
  "Razonamiento": [
    "El usuario alumno ejecuta el comando 'cat' sobre un archivo de credenciales (passwd_changes)",
    "Luego, el mismo usuario intenta modificar sus privilegios sudoers con el comando 'sudo' sin éxito (sudoers_changes: success=no)",
    "El usuario alumno vuelve a intentar modificar sus privilegios sudoers con el comando 'sudo', pero esta vez con éxito (sudoers_changes: success=yes)",
    "Después de obtener los privilegios sudo, el usuario alumno cambia las credenciales del sistema operativo (passwd_changes)",
    "Finalmente, el usuario alumno ejecuta el comando 'ausearch' con permisos de root (root_commands)",
  ],
  "Riesgo": "ALTO"
}
[=] Tiempo de procesamiento: 1 min 41.22 s (101.22 segundos totales)
```

```
[+] Procesando bloque: V-NRM-03.txt | Ground Truth: BAJO
[*] Enviando payload a la GPU del Host (Temperatura: 0.2)...
[=] RESPUESTA DEL MODELO:
{
  "Razonamiento": [
    "Se observa una serie de comandos ejecutados por el usuario 'alumno' con privilegios elevados a través del uso de sudo.",
    "El usuario intentó cambiar sus credenciales de acceso, lo que es un indicador de actividad sospechosa.",
    "No se observa ninguna escalada de privilegios ni modificación de archivos sensibles.",
    "La denegación de permisos por sudo no fue exitosa.",
  ],
  "Riesgo": "MEDIO"
}
[=] Tiempo de procesamiento: 3 min 10.79 s (190.79 segundos totales)
```

Phi-3 Mini:

```
[+] Procesando bloque: V-NRM-01.txt | Ground Truth: BAJO
[*] Enviando payload a la GPU del Host (Temperatura: 0.2)...
[=] RESPUESTA DEL MODELO:
{
  "Razonamiento": "El log muestra una secuencia sospechosa de eventos que indican un posible ataque coordinado. El usuario 'alumno' inició con comandos interactivos estándar como ls, seguidos por múltiples intentos exitosos para ejecutar sudo y cambios en los privilegios del sistema utilizando el comando sudo sin denegación previa (indicativo de una secuencia sospechosa). Esto se repite varias veces con diferentes comandos, lo que sugiere un comportamiento no autorizado. Además, hay múltiples intentos exitosos para cambiar los privilegios del usuario a root y luego al menos uno fallido (exit=6), posiblemente debido a una contraseña incorrecta o medidas de seguridad adicionales en respuesta.",
  "Riesgo": "ALTO"
}
[=] Tiempo de procesamiento: 2 min 26.73 s (146.73 segundos totales)

[+] Procesando bloque: V-NRM-02.txt | Ground Truth: BAJO
[*] Enviando payload a la GPU del Host (Temperatura: 0.2)...
[=] RESPUESTA DEL MODELO:
{
  "Razonamiento": "El registro inicial muestra una ejecución del comando 'cat' por un usuario con UID y GID no especificados como alumno. Luego se observa que el mismo usuario realiza múltiples intentos de usar sudo, todos exitosos, lo cual es inusual ya que los comandos suelen fallar si la sesión del usuario no tiene privilegio suficientes para ejecutar 'sudo'. Esto podría indicar un comportamiento sospechoso. Posteriormente se registran cambios en las bases de datos de credenciales, primero como root y luego con el mismo UID que antes (alumno), lo cual sugiere una posible escalada de privilegios o manipulación de credenciales. Finalmente, 'ausearch' se ejecuta para buscar comandos relacionados a los usuarios root, indicando un intento de investigar actividades sospechosas previamente registradas.",
  "Riesgo": "ALTO"
}
[=] Tiempo de procesamiento: 3 min 29.66 s (209.66 segundos totales)

[+] Procesando bloque: V-NRM-03.txt | Ground Truth: BAJO
[*] Enviando payload a la GPU del Host (Temperatura: 0.2)...
[=] RESPUESTA DEL MODELO:
{
  "Razonamiento": "La ventana temporal muestra una serie de auditorías que comienzan con comandos ejecutados por el usuario root como 'cut' y 'grep', seguidos por la actualización del motivo (update-motd-reb) e interacción con un proceso Dash, luego se registra varias llamadas a syscall execve para acceder al sistema de archivos. Posteriormente, hay múltiples intentos exitosos y denegados por sudo que resultan en cambios en los permisos del usuario 'alumno' (UID=1000) con el SGID='root', lo cual es altamente sospechoso. Finalmente, se registran varias llamadas a syscall openat relacionadas con passwd_changes y sudoers_changes que indican cambios en las credenciales del sistema operativo y privilegios de usuario.",
  "Riesgo": "ALTO"
}
[=] Tiempo de procesamiento: 3 min 7.93 s (187.93 segundos totales)
```